Exercise 2

SQL Exercise : Aggregate Functions and Grouping

Database : Employees table

1. COUNT () Function

```
SELECT COUNT(DISTINCT(employee_id)) AS total_number
OF employees
FROM employees
```

Number_of_employees
10

2. Sum () Function

```
SELECT Sum(Salary) AS total_IT_Salary_Paid

FROM employees
WHERE department = 'IT';
```

3. AVG () Function

Department	Salary
IT	55000 + 5300 + 62000 + 50 000 = 220 000

3. AVG() Function

```
SELECT AVG(salary) AS average_salary_HR
FROM employees
WHERE department = 'HR';
```

Department_HR	AVG salary
HR	(51 000 + 48000) / 2 employees = 99 000 / 2 = 49 500 →

4. MIN() and MAX Functions

```
SELECT
    MAX(salary) AS highest_salary
    MIN(salary) AS lowest_salary
FROM employees;
```

MAX_Salary	MIN_salary
62 000	48 000

5. GROUP BY Statement

```
SELECT department,
    SUM(salary) AS total_salary_paid
FROM employees
GROUP BY department
```

Department	Total salary Per department
IT	220 000
HR	99 000
Finance	119 000
Marketing	105 000

6. GROUP BY and COUNT()

```
SELECT city,
       COUNT(*) AS employee_count
FROM employees
GROUP BY city
```

City	Employees (no of)
New York	2
Chicago	3
Los Angeles	2
San Francisco	2
Houston	1

7. GROUP BY and ORDER BY

```
SELECT department,
       AVG(Salary) AS avg_salary,
       COUNT(* (no of employees)) AS employee_count
FROM employees
GROUP BY department
ORDER BY avg_salary DESC;
```

no of employees	Department	Average salary DESC
4	IT	220 000/4 = 55 000
2	HR	99 000/2 = 49 500
2	Finance	119 000/2 = 59 500
2	Marketing	105 000/2 = 52500

NB (DESC) = Finance = 59 500, IT = 55 000, marketing = 52 500, HR = 49 500

(table is not in descending order)

8. Having Clause

```
SELECT department,
       SUM(salary) AS total_salary_paid
FROM employees

GROUP BY department
HAVING SUM(amount) > 100 000;
```

Department	Total Salary Paid > 100000
IT	220 000
Finance	119 000
Marketing	105 000

9. Combining GROUP BY, HAVING, and ORDER BY

```
SELECT city, COUNT(employee no) AS employee_count;
FROM employees

GROUP BY city
ORDER BY employee_count DESC
HAVING COUNT(no of employees) > 1
```

City	no of employees (DESC)
Chicago	3
New York	2
Los Angeles	2
San Francisco	2

10. Combining Aggregate Functions

```
SELECT department,
       AVG(salary) AS average_salary
       COUNT(employees) AS employee_count
FROM employees
GROUP BY Department
ORDER BY avg_salary DESC
Limit 1;
```

Department	Total Salary Paid
IT	22 000

10. Combining Aggregate Functions

```
SELECT department,
       AVG(salary) AS average_salary
       COUNT(employees) AS employee_count
FROM employees
GROUP BY Department
ORDER BY avg_salary DESC
LIMIT 1;
```

Department	Total Salary Paid
IT	22 000

Amount to be deducted in the future

Details	CA	LB	TD	
Prepaid expense	20 000	—	7.	

Statement

CT department

yees

3. Se

first_name,
Last_name

FROM employees
ORDER BY Salary DESC;

4. SELECT *
FROM employees
ORDER BY Salary DESC
LIMIT 3

6. SELECT *
FROM employees

WHERE department = 'Finance'
AND Salary > ~~58 000~~ 60 000

5. SELECT *
FROM employees
WHERE department = 'IT';

Exercise 1 SQL Fundamentals

1. Select statement

```
SELECT *
FROM employees
```

2. SELECT DISTINCT Statement

```
SELECT DISTINCT department
FROM employees
```

3. SELECT

```
SELECT first_name,
       last_name
FROM employees
ORDER BY Salary DESC;
```

4.

```
SELECT *
FROM employees
ORDER BY Salary DESC
LIMIT 3
```

6.

```
SELECT *
FROM employees

WHERE department = 'Finance'
AND Salary > ~~55 000~~ 60 000
```

5.

```
SELECT *
FROM employees
WHERE department = 'IT';
```

7. SELECT employee_id
department
FROM employees
WHERE department = 'HR' OR
department = 'marketing';

8. SELECT department
FROM employees
WHERE ~~V~~ department NOT ~~#~~ = IT;
NOT <> - not equal

9. SELECT department
FROM employees
WHERE department IN ('HR, IT, Finance');

10. SELECT *
FROM employees
WHERE department = 'IT',
AND Salary > ~~50 000~~ 65000
AND City = ~~'New York'~~; 'Johannesburg';

---

11. SELECT first_name,
last_name
FROM employees
WHERE department = ('Finance, marketing')
AND Salary > 52 000
ORDER BY Salary DESC

12. SELECT DISTINCT city
FROM employees
WHERE department IN ('Finance, marketing');

13. SELECT *
FROM employees
WHERE department NOT IN ('Finance')
AND Salary > 50 000
ORDER BY hire_date ASC

14. SELECT *
FROM employees
City = 'Chicago, ~~Los Angeles~~'
OR city = 'Los Angeles'
AND Department IN ('IT, OR marketing
~~ASC OR department~~
LIMIT 3;

Exercise 3 - SQL CASE Statements

1. SELECT Product name,
Price

```
CASE
    WHEN Price > 1000 THEN 'Expensive'
    WHEN Price BETWEEN 100 AND 1000
     THEN 'Mid-range'
    WHEN Price < 100 THEN 'BUDGET'
END AS Price category
 FROM Products
```

Product_name	Price	Price Category
Laptop	1200	Expensive
Phone	800	Mid-range
Keyboard	45	Budget
Minotor	300	Mid_range
Mouse	25	Budget

2. SELECT Customer name,
amount

```
CASE
    WHEN amount >= 1000 THEN 'High Value'
    WHEN amount BETWEEN *500 to 999,99
     THEN 'medium value'
    WHEN amount < 500 THEN 'Low value'
END AS order value category
FROM orders
```

Cust_name	Amount	Order_value category
Alice	150	Low value
Bob	560	Medium value
Charlie	999,99	Medium value
Diana	45,50	Low value
Ethan	12000	High value

3. SELECT emp_name
Salary

CASE

~~WHEN~~ THEN

IF in IT and Salary > 80000 → Senior IT

If in HR and Salary > 50000 THEN

Experienced HR

Otherwise → Staff

emp_name	Department	Salary	Position level
John	IT	85000	Senior IT
Sara	HR	60000	Experienced HR
Mark	IT	75000	Staff
Lucy	Finance	95000	Staff
Tom	HR	55000	Staff

4.

SELECT Student_name,
Score

CASE
WHEN SCORE ≥ 90 THEN A
WHEN SCORE BETWEEN 80 ~~TO~~ AND 89 THEN B
WHEN SCORE BETWEEN 70 ~~AND~~ AND 79 THEN C
WHEN Score BETWEEN 60 AND 69 THEN D
WHEN Score ~~B~~ < 60 THEN F

OUTPUT

Student_name	Score	Grade
Anna	92	A
Ben	76	C
Carg	59	F
David	83	B
Ella	68	D

5. SELECT delivery_id,
delivery_time_minutes

CASE
WHEN minutes ≤ 30 THEN 'FAST'
WHEN minutes BETWEEN 30 AND 60 THEN 'ON TIME'
WHEN minutes 60 THEN 'LATE'

Output

delivery_id	delivery_time_minutes	Performance
1	45	On Time
2	80	LATE
3	30	Fast
4	65	LATE
5	100	LATE

6. SELECT Issue_type,
Priority

CASE
WHEN Priority 3 THEN High
WHEN Priority 2 THEN Medium
WHEN Priority 1 THEN LOW

Issue_type	Priority	Priority_label
Login issue	1	Low
Server down	3	High
Slow System	2	Medium
Email error	2	Medium
Password reset	1	Low

7. SELECT atterdance_percentage
attendance_status

CASE
WHEN atterdance % ≥ 90% THEN 'Excellent'
WHEN atterdance % BETWEEN 75 AND 89%
THEN 'Good'
WHEN atterdance % ≤ 75% THEN 'Needs Improvement'

Output

Student_id	days present attendance	total_days	Attendance %	Status
1	45	50	90%	Excellent
2	30	50	60%	Improvement Needs
3	48	50	96%	Excellent
4	25	50	50%	Improvement Needs
5	50	50	100%	Exellent

8.
```
SELECT stock_qty,
       Stock_status
CASE
    WHEN STOCK STATUS != 0 THEN 'Out of Stock'
    WHEN Stock Status BETWEEN 1 AND 5 THEN
    'Low on Stock'
  WHEN Stock Status 5 THEN 'In Stock'
```

Output

Product_id	stock_qty	stock_status
1	5	In stock
2	0	Out of stock
3	25	In stock
4	10	In Stock
5	3	Low Stock

9.
```
SELECT subject,
       enrolled_students
CASE
    WHEN Size > 25 THEN 'Large'
    WHEN SIZE BETWEEN 10 AND 24 THEN
    'MEDIUM'
  WHEN SIZE < 10 THEN 'SMALL'
```

Output

Subject	Enrolled_Std	class_size_category
Math	30	Large
English	25	Large
Science	15	Medium
Art	5	Small
History	20	Medium

10. SELECT Amount
discount_elegibility
CASE
WHEN If payment_method = Cash and amount ≥ 200 THEN 'Eligible for DISCOUNT'

WHEN Otherwise THEN 'NOT Eligible'

Output

Payment Id	Payment_method	Amount	DISCOUNT_eligibi
1	Card	50.00	NOT Eligible
2	Cash	200.00	Eligible for Discoun
3	Card	150.00	NOT Eligible
4	PayPal	75.00	NOT Eligible
5	Cash	300.00	Eligible FOR Discount